Experiences with Technical Debt and Management Strategies in Production Systems Engineering

Laura Waltersdorfer
SBA Research
Christian Doppler Lab CDL-SQI
LWaltersdorfer@sba-research.org

Felix Rinker
Christian Doppler Lab CDL-SQI, ISE
Technische Universität Wien, Austria
felix.rinker@tuwien.ac.at

Lukas Kathrein
Christian Doppler Lab CDL-SQI, ISE
Technische Universität Wien, Austria
lukas.kathrein@tuwien.ac.at

Stefan Biffl
Inst. of Information Systems Eng.
Technische Universität Wien, Austria
stefan.biffl@tuwien.ac.at

ABSTRACT

Technical Debt (TD) has proven to be a suitable communication concept for software-intensive contexts to raise awareness regarding longterm negative effects of deviations from standards and guidelines. TD has also been introduced to systems engineering domain, to communicate design shortcomings in long-running, software-assisted systems. We analysed potential TD in the engineering data exchange for production system engineering. Similar to requirements engineering in software-intensive systems, data exchange in the design phase plays an integral part in Software Engineering (SE) for Production Systems Engineering: Specifications, and physical logic have to be derived from heterogeneous plant models or parameter tables designed by different stakeholders. However, traditional procedures and inadequate tool support lead to inefficient data extraction and integration. We identified debt arising from knowledge representation, data model and the exchange process. The refinement validation of identified TD was achieved through semi-structured interviews with representatives in two analysed companies. In an online survey with ten participants from an industrial consortium we evaluated whether the identified TD concepts also applied to other companies, which is true for the majority of TD. Furthermore, we discuss promising TD management strategies to repay and manage negative effects and the accumulation of additional debt, such as improved communication, test-driven model engineering and visualisation of engineering models.

CCS CONCEPTS

• **Computer systems organization → Maintainability and maintenance**; • **General and reference → Empirical studies**; **General conference proceedings**; • **Software and its engineering →** *Software design tradeoffs*; **Software implementation planning**; **Error handling and recovery**; *Maintaining software.*

TechDebt '20, October 8–9, 2020, Seoul, Republic of Korea

ACM ISBN 978-1-4503-7960-1/20/05…$15.00
https://doi.org/10.1145/3387906.3388627

KEYWORDS

Technical Debt, Industrial Informatics, Technical Debt in Systems Engineering, Technical Debt Management in Systems Engineering

ACM Reference Format:
Laura Waltersdorfer, Felix Rinker, Lukas Kathrein, and Stefan Biffl. 2020. Experiences with Technical Debt and Management Strategies in Production Systems Engineering. In *International Conference on Technical Debt (TechDebt '20), October 8–9, 2020, Seoul, Republic of Korea.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3387906.3388627

1 INTRODUCTION

Insufficient requirements specification, undocumented functionality and unstructured communication can lead current Software Engineering (SE) projects to failure and create Technical Debt (TD) [19] for future projects. Production Systems Engineering (PSE) shares similar challenges, since software and hardware have to be integrated to achieve the expected results: Wrong construction parts, ill-measured cabling or missing software functionality at the construction site can provoke delay and high financial costs [42].

Software supports the achievement of production goals of a working cell in production systems in terms of speed or output by controlling robotic arms, pumps or transportation systems in an efficient way. However, in traditional industrial projects SE contributions are made in the final phases of the design phase based on the engineering information derived from other disciplines and thus, have to rely on the quality and availability of exchanged data between engineering disciplines, such as mechanical, electrical or pneumatics [6].

These artefacts are enriched with more details over the course of the project by various stakeholders in a so-called round-trip engineering process [11], depicting relationships between components and dependencies: Mechanical engineers specify the dimensions of a coil car which is moved with a certain motor and power supply calculated by the electrical engineer, and also influenced by the level of fluidics, such as oil or compressed air for example for the brakes.

If one component is modified or removed from the design, this change might trigger changes in other disciplines, as well, which can lead to TD by deviating from best practices in data integration [27] or industry guidelines [39]. Thus, we have to observe data exchange between all disciplines and cannot only focus on SE to improve the overall process and manage TD [3].

Due to historic and technological reasons, a variety of data formats and software tools are used in the production systems design phase [37]. Common Computer-Aided Design (CAD) tools result in over ten different native formats and Excel spreadsheets are common for parts lists [37]. Such a heterogeneous environment makes the validation and verification of exchanged artefacts cumbersome and error-prone.

Manual analysis and correction has to be done, which is difficult because of the size of the files and the low level human-readability, which can lead to costly TD in the project. Automation engineers aim to simulate the entire system to check for safety risks or consistency errors in the design. However, these simulation models are constructed on the parameters extracted from engineering data to imitate the systems behaviour. Thus, the quality of exchanged engineering artefacts has an influence on the coherence and correctness of system simulations and thus, its validation.

In this paper, we report on our experiences of analysing TD in the data exchange process of Production Systems Engineering based on a cross-case study in two PSE companies. We evaluated our findings through expert interviews and an online survey to gain additional insights. Furthermore, we discuss various debt management and repayment strategies that we conducted during our projects to manage and repay certain debts in the industrial context.

The remainder of this paper is structured as follows: Section 2 presents related work on general characteristics and procedure of engineering projects, as well as TD in SE and TD in systems engineering. Section 3 presents the context of the conducted case studies and our research approach. We describe our TD framework in Section 4 and present the results of the evaluation in Section 5. We discuss our findings and related limitations in Section 6. Finally, Section 7 summarizes our results and discusses research directions and future work.

2 RELATED WORK

In this section, we discuss relevant works on systems engineering, specifically data exchange in Production Systems Engineering (PSE) and on Technical Debt (TD).

2.1 Systems Engineering

Systems engineering is characterised by large and long-running systems needing and producing a vast amount of data. In [34], the authors explain which research gaps exist in reducing the Information technology (IT) complexity in the manufacturing industry. One of the main steps to reduce complexity and insufficient management of the Information systems (IS) landscape is the introduction of a software-defined platform. Hummel et al. described challenges for data-intensive contexts such as inadequate debugging and error -tracing methods and tools, difficult visualisation and explainability or complex data processing [15]. The concept *single source of truth* is mentioned to manage such complexity to provide a common knowledge bases integrating and translating all heterogeneous views and concepts within the organisation. [12] describes such an single source of truth approach for the modeling of the flight system design of the Europa Project by National Aeronautics and Space Administration (NASA) to ensure the correctness of the model and to enable making queries on the information stored in the models. The authors identified both technical and social challenges associated with the approach.

PSE involves the collaboration of multiple engineering disciplines, such as mechanical, electric and pneumatic engineering. Each discipline has its own view on the system, describing discipline-specific components, which can also influence the design and functioning of other disciplines.

Date Exchange in (Production) Systems Engineering. Common project phases in PSE projects are acquisition, planning, realisation and commissioning [40].

Data exchange plays an essential part in all phases, and is a complicated activity. Figure 1 illustrates the use case in a highly simplified way: Various stakeholders, such as engineering workgroups, system planners, production optimizers, project managers, simulation engineers and automation engineers collaborate by exchanging artefacts in a plethora of formats (e.g. Computer-Aided Design (CAD), Comma Separated Value (CSV), Portable Document Format (PDF)) to design a coherent system over the course of a PSE project. Data exchange is usually conducted via mail in an artefact-based way. Additional information is provided through unstructured communication channels, face-to-face or via telephone. A more detailed explanation of the simulation engineer and project manager use case is provided in [6].

Due to time constraints and market pressure, engineers have to design in parallel. Moreover, planning and commissioning of systems happen in parallel [37], meaning that parts of the systems are already ordered and in construction, while others yet have to be planned. Due to compatibility issues between different disciplines and domain-specific tools, artefacts are often exchanged in PDF or spreadsheets, which has its disadvantages, such as limited accessibility and semantic structure [29]. Therefore, consistency is important for the successful design of production systems, but hard to to comply without adequate tool and process support [41]. Extensible Markup Language (XML)-based formats are also common, tailored to the application context, such as AutomationML (AML) [16] in the production systems context or Railway Markup Lanuage (railML) in the railway data exchange [26]. However, the implementation of new languages only alleviates some negative characteristics of the systems engineering process: Process and tools are still adapted to traditional data exchange.

Holistic approaches such as Building Information Model (BIM) show that multiple aspects of the engineering process have to be adapted and transformed to benefit from improved characteristics of the overall process in the present and future [2]. In Software Engineering (SE), agile approaches are popular and not only induce technological but also massive cultural change [35].

2.2 Technical Debt

This subsection gives an overview about relevant work in TD in SE, in systems engineering and TD management strategies.

Technical Debt in SE. TD is a well-established research topic in SE [19] to illustrate and communicate negative long term effects in the design and implementation of software systems. Martini et al. have shown that approximately a quarter of time is spent for repayment and management of accumulated TD [21]. Various types of debt

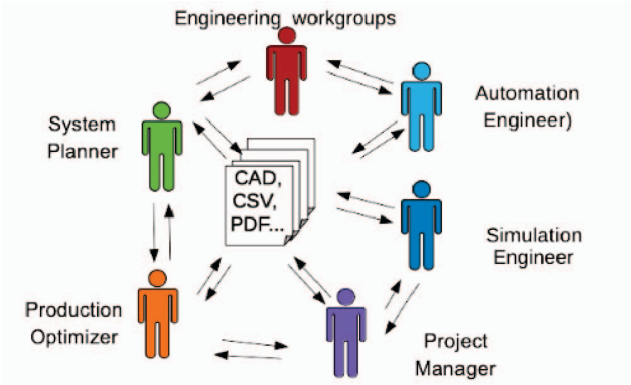

Figure 1: Roundtrip Engineering Use Case based on [6]

have been detected, as Li et al. collect ten types, ranging from test to documentation and infrastructure debt [19].

Common causes for TD vary, however oftentimes time pressure, parallel development, human factor, changing or undocumented customer expectations [23]. Moreover, also non-technical factors can also influence the accumulation of TD, such as undervaluing TD management, miscommunication about strategic goals, conflicting management styles and systems [9].

Technical Debt in Systems Engineering. Vogel-Heuser et al. has introduced the concept to automated production systems to show the applicability and the need to adapt SE methods to the domain [42]. Besker et al. have analysed in a case study with automated production systems that compared to software-intensive companies even more development time (in average 32% vs. 25%) is used for TD repayment and management in industrial settings [3]. Dong et al. have shown that the electrical domain takes out significant TD, with SE bearing negative effects [10].

Technical Debt Management. The repayment and management of TD in software-intensive environments is also an important topic in the research: The audit and identification of TD is considered to be a vital first step for its later repayment and management. Guo *et al.* [14] showed that quantifying and managing debt may incur higher startup costs in the beginning, but that costs decline to reasonable levels after having taken the starting effort. In [25] Morgenthaler et al. discuss strategies and three underlying principles managing TD at Google: Automation, analysing TD, and introducing stricter checks into processes are major factors as their management approach for build debt. In [44] the authors present a three level framework for SE teams for TD management: Organisations can adapt the suggestions related to the level of TD management they want to conduct (from unorganised to organised).

3 RESEARCH METHODOLOGY

In this section we discuss the research approach and the case study context.

3.1 Research Context

The primary goal of our research was to support the digitisation of the Production Systems Engineering (PSE) to optimise the engineering data exchange and enable the transformation towards Cyber-physical production system (CPPS). Over the course of our research project we identified issues and challenges that seemed familiar to Software Engineering (SE) and Technical Debt (TD), such as limited quality assurance and high error rate for manual tasks. Thus, our aim was to test whether TD can be identified in the companies to extend traditional TD models [19, 38] and frameworks to the industrial domain by analysing the engineering data exchange.

Company Partners. Both analysed companies have their headquarters in the German-speaking region: Company A is a global leading metal company with over 10 000 employees internationally. Company B is slightly smaller with around 2000 employees, focusing on product and high performance automation. Although, their products differ, the process of data exchange is similar as described in the following.

Motivational Use Case. As described in Section 2, engineering data exchange is a vital task for the successful engineering of Production System (PS). In Figure 1 a selection of roles that can be involved in the data exchange is depicted. Certain roles need a discipline-specific view on the exchanged data, such as mechanical, electrical or fluidic engineers for examples.

Other stakeholders, such as automation engineers and project managers need discipline-neutral views to evaluate the overall project status or test the entire system virtually for functionality. Due to the multi-time exchange and dependent designs on other disciplines, consistency and versioning would be essential, however are not always covered. Thus, we analysed this underlying use case for the scope of our TD analysis to derive improvement techniques.

3.2 Research Methods

The primary goal of the research is to find out whether TD can be identified in this motivational use case, engineering data exchange and how SE methods can be adapted to this specific context, as proposed in [42] to manage and alleviate the negative TD effects.

3.2.1 Case Study Design & Planning. We formulated the hypothesis that in middle- and large-size PSE companies certain best practices [27, 40], concerning the engineering data exchange are not adhered to. This deviation of best practices might lead to TD. Because of the heterogeneity of data formats, unstructured data and discipline-specific views in the PSE domain, an automated analysis was not feasible and thus we decided to conduct a case study according to [33] and treatments according to the design science cycle [43].

We investigated the problem context through archival data analysis and literature analysis, proposed a treatment design (technical debt framework building blocks and TD management measures), and validated our design through a mix of qualitative and quantitative measures such as interviews and surveys. The materials can be

accessed online[1]. Treatment implementations are currently under investigation through improvement initiatives in the companies.

3.2.2 Data Collection. We combined archival data analysis, with collecting new qualitative and quantitative data. In prior work over one year, we analysed common use cases to derive requirements and challenges towards a round-trip engineering (RTE) process in two companies [6, 7, 18]. The archival data analysis, combined with additional provided data from the partner companies (e.g. spreadsheets) was the basis for our preliminary TD analysis on different TD aspects and submodels: (a) [4] explores the influence of project-dependent factors and causes on the overall engineering data exchange, (b) [5] subsequently investigates the role and impact of missing/underrepresented knowledge in the data exchange and (c) while [8] discusses TD focusing on engineering description languages. These works are the foundation for the proposed framework in this paper and were refined by more analysis and the results of interview and survey. The three categories TD data model, knowledge representation and exchange process seemed to be the most pressing ones after analysing the process in both companies.

For validation purposes, we conducted two semi-structured interviews with project managers from both respective companies to deepen our understanding regrading causes and effects of TD in the engineering data exchange and test our hypotheses regarding TD. The interview guideline was devised after literature research on TD and the use case workshops. To improve the understandability, the guideline was tested and refined with senior researchers and members of the research team. Furthermore, we conducted an online survey in the context of an engineers member association to further validate whether the identified TD items were also relevant for other companies. Again, the questions were tested and improved in the research team to increase understandability and reduce bias. Due to the limited scope of the survey, causes were not questioned. The focus was laid on the TD categories of the framework and experienced causes in a previous project.

3.2.3 Data Analysis. We revisited our works [5, 6, 8] to analyze commonalities and differences of the results from the two partner companies and synthesize the results into a TD framework in Section 4 for multi-disciplinary data exchange in PSE. Based on the interviews and provided data, we designed and refined the TD framework presented in Section 4. The framework itself was not questioned explicitly in the interviews, since the framework was based on the workshop data and process analysis of the two companies and therefore would be a circular argument. The framework was only refined after an open discussion with the domain experts in terms of causes and effects and whether statements in favor of the identified TD items were found in the arguments by the domain exerts. Furthermore, we analysed the data generated by the interviews to find out more regarding causes and effects of TD in PSE discussed in Section 5. We mapped relevant statements to known TD characteristics found in literature [19, 38]. The survey results were analysed, whether the identified TD categories also apply to other companies and increase the validity of the framework, explained in Section 5. Potential management techniques were also discussed within the interviews, some of them also explored in Section 6.

[1]https://cdl-sqi.ifs.tuwien.ac.at/techdebt2020

4 TECHNICAL DEBT FRAMEWORK FOR DATA EXCHANGE IN PRODUCTION SYSTEMS ENGINEERING

Our Technical Debt (TD) framework for data exchange Production Systems Engineering (PSE), shown in Figure 2, consists of three major pillars: TD causes in the left, TD categories and effects. We have grouped the causes into four groups, business, organisational, process and technology for better overview. In the middle are three TD categories: *Data Model*, *Knowledge Representation* and *Exchange Process*, under which we identified specific TD items. In the following we describe the identified TD categories and items to provide context information, the debt itself, an illustrative example and the principal. On the right we have high-level TD effects observed in our analysis.

4.1 Technical Debt Data Model

This TD type is concerned with the data model.

Insufficient Data Model Documentation.

- *Context:* Data consumers need not only the requested engineering data, but also the underlying data model in order to understand the provided data.
- *Technical Debt:* According to the descriptions of participants, the form of data models between different workgroups can differ: In some cases, only textual description was provided, in other cases a modelled data model was given. Moreover, the scope and quality of data models were different: Data models were not updated regularly, or even complete, because data exchange is not considered of major importance for the overall process.
- *Example:* For mechanical engineering, components lists from suppliers are utilized for documentation, meaning that essential values have to be looked up in non-searchable Portable Document Format (PDF) tables.
- *Principal:* A data model format should be agreed upon, so no confusions arise between workgroups. Updates and checks on the data model by experienced domain experts should be scheduled and considered a meaningful task.

Insufficient Product Process Resources (PPR) Model Documentation.

- *Context:* PPR is a means of knowledge representation to improve engineering knowledge representation and facilitate optimisation efforts for software engineers [30]. The increased complexity of production processes requires the knowledge of dependencies between the different production aspects.
- *Technical Debt:* However, in the case study context process designers either tried to save time by just providing default values or none at all. Furthermore, there is no tool support for modeling PPR information explicitly, which makes it harder to detect.

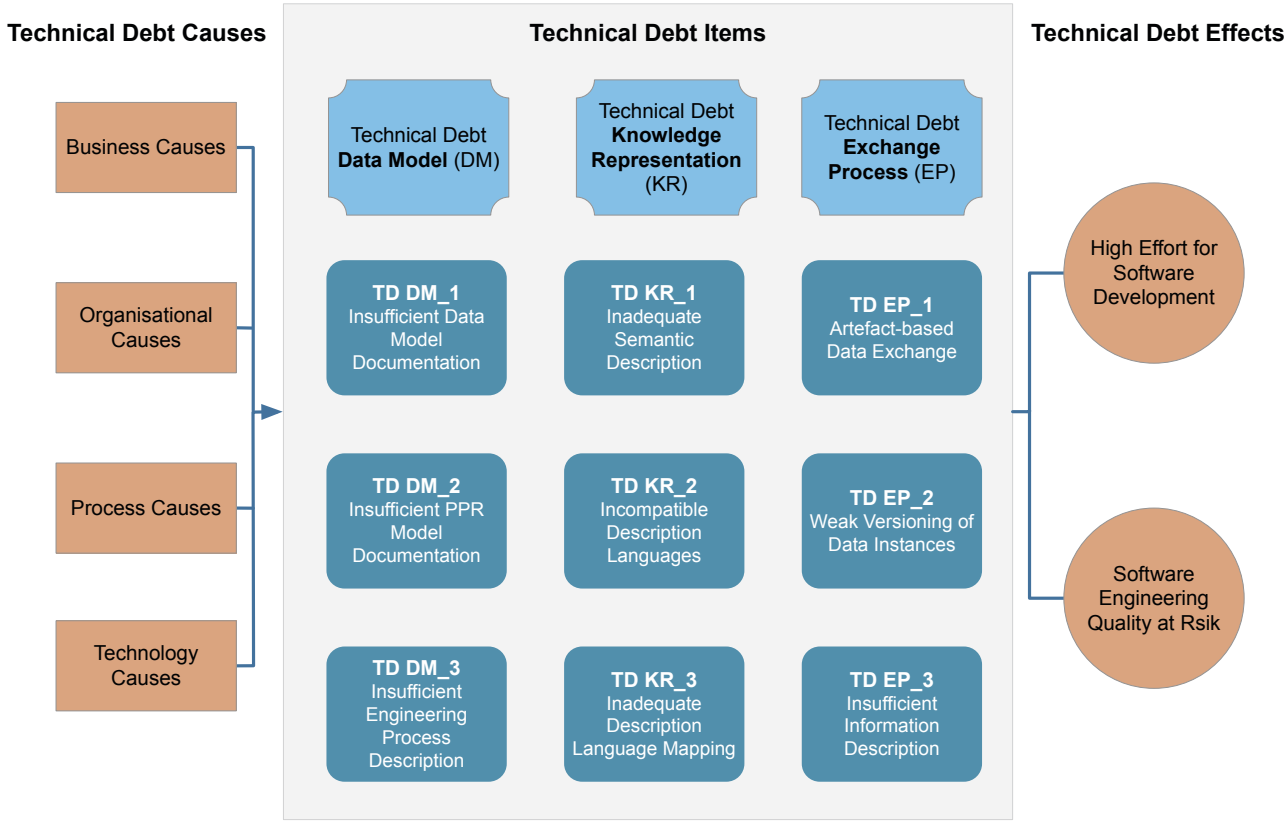

Figure 2: Technical Debt Framework in Production Systems Engineering based on [4, 5, 8]

- *Example:* Calibration points of welding machines for products are nowhere defined, instead they have to be tried out or deduced from past projects. Design decisions are not traceable and based on experience. This behavior is error-prone and time-consuming.
- *Principal:* A format to model PPR information should be agreed upon. Due to the novelty of this measure, tool support is still in its infancy and should be implemented as a long term measure.

Insufficient Engineering Process Definition.

- *Context:* Engineering processes depend on the characteristics of a project and are flexible: Involved disciplines, customer requirements, physical constraints and other factors can influence the sequence and structure of process steps.
- *Technical Debt:* Domain experts base designs on their personal experience, which are not explicitly modelled. Another source are designs from past projects, which are only locally saved or implicitly modelled. There is no formal description of discipline- specific engineering process steps and of the collaboration tasks. This prevents systematic or any quality assurance without the specification of standard procedures.
- *Example:* The engineering process is a highly creative and innovative process. Thus, there are no fixed tasks or checklists, what needs to be done for specific engineers across workgroups.
- *Principal:* Checklists for the most important process factors should exist, and should be extended after every completed project. The conceptualisation of these checklists should be incorporated into the usual post-project activities as lessons learned. However, it is important to note that the implementation should not constrain the designing creativity, but rather should provide a guiding framework.

4.2 Technical Debt Knowledge Representation

This TD type is concerned with (engineering) knowledge representation, in our case we focused on description languages.

Inadequate Semantic Description.

- *Context:* The semantic description of a data instance should provide details about the meaning of values or data.
- *Technical Debt:* It is considered inadequate for data exchange, if the description is incomplete or incorrect. This hinders the automated access and transformation. Flexible structures of spreadsheet for example allowed data providers to encode information in different locations.

- *Example:* A mechanical engineer might define *coil car* in the documentation, whereas in electrical engineering the abstract specification defines it as a *machine*.
- *Principal:* There should be a minimal agreed semantic description for stakeholders to easily detect essential data and minimise integration effort.

Incompatible Description Languages.

- *Context:* Each engineering discipline has its specific concepts and views, and thus requires a different set of notations. The translation to other disciplines is not always explicitly modeled and communicated.
- *Technical Debt:* Description languages do not describe semantically equivalent concepts or cannot be automatically transformed. Domain experts experience difficulties to integrate foreign languages into their concepts and models, which can lead to wrong translations or loss of information.
- *Example:* CAD formats tend to raise incompatibility issues due to the proprietary data format and can be only edited by discipline-specific tools.
- *Principal:* Data providers and data consumers have to agree upon a certain translation of description languages to prevent a confusion of concepts between workgroups.

Inadequate Description Language Mapping.

- *Context:* In this case the description languages describe semantically equivalent concepts. But, the automatic transformation of semantically equivalent concepts between these description languages is not possible.
- *Technical Debt:* Domain experts still need to manually transform concepts into their concept language. Manual transformation leads to human errors and increases the cost and time: In big data sets, mistakes are common and typos could lead to critical errors for example. Corrections take time and can prevent timely progress of the project.
- *Example:* In the mechanical view a machine has several interfaces for power and fluids, which need to be mapped to the electrical and fluidic view. So far only spatial information is provided for mapping purposes, since plugs have no unique id for example. Thus, domain-specific expertise is needed to translate knowledge how the plugin is depicted in the CAD drawing.
- *Principal:* The automatic translation and integration of description languages can prevent manual transformation errors and reduce time.

4.3 Technical Debt Exchange Process

This TD type is concerned with the data exchange process.

Artefact-based Data Exchange.

- *Context:* Engineering data exchange is considered a cost factor rather than a value-creating activity. Thus, domain experts try to reduce effort by exchanging artefacts conveniently. Instead of providing the essential minimum of information, most data providers export all their engineering data conveniently from their discipline-specific tool. In the attempt to save time, data consumers provide all the data they have, enriched with unnecessary information that is not needed and can confuse receiving data consumers.
- *Technical Debt:* Software engineers and all other data consumers have to assess the complete artefact provided by the domain expert, for example a spreadsheet with thousands or even millions of values. The data consumer would only need a smaller amount of values, however he needs significantly more time and effort to import and decode the data instance: In the case of missing documentation, he needs to contact the other expert, to find out where specific values are located. This could be described as contagious debt leading to vicious cycles, as in [22]. Furthermore, the artefacts need to be accessed individually, if changes throughout its lifecycle need to be investigated.
- *Example:* Mechanic engineer sends an unedited export of component list in spreadsheet format to electric engineer, who only needs a small subset of these values, but needs to search these specific values.
- *Principal:* Agreeing on the needed minimum of information and exchanging engineering models in a neutral format instead of engineering artefacts encoded in proprietary formats would remove or lower the TD in this case for the software engineer and other data consumers.

Weak Versioning of Data Instances.

- *Context:* Due to the parallel planning activities in different work groups, multiple versions of the same data instance float around. Change tracking is also only achieved manually by comparing different data dumps, and makes it therefore easy to overlook minor changes. This debt is also connected and correlated to artefact-based data exchange.
- *Technical Debt:* Versioning is a crucial improvement in Software Engineering (SE). The non-existence of a versioning mechanism leads to multiple challenges: Data consumers are unaware about new data deliveries without notification by the data provider. Changes are not easily detected and design decisions get lost over the course of the project.
- *Example:* The floor plan is provided to all engineering workgroups. Engineers access the file and make local changes on their machines. These local changes are sent to single engineers, but not to all stakeholders, since there is no overview over all relevant parties. In this context, these local modifications can get lost, insignificant or conflicted due to changes by others.
- *Principal:* Data consumers would benefit from a central repository similar to SE practice, a notification could be sent to domain experts waiting for certain data. Change tracking (diff algorithms) could be integrated into the central data hub to ease the detection of differences.

Insufficient Information Description.

- *Context:* The engineering data exchange process is highly unstructured. Implicit structures between workgroups exist, but are not explicitly modelled. Experts only model the scope of their artefacts, but not the connection to the rest of the system.

- *Technical Debt:* New staff, or software engineers switching to different workgroups, have to find out by themselves how to access information and whom to inform. Also in the case of changing responsibilities, data consumers are not automatically aware who is in charge or whom to contact.
- *Example:* Date exchange is highly dependent on the involved people. There is no clear communication approach, but rather a broadcast approach, which can also be a security challenge. In case of absence of significant points of communication (project manager, or experienced engineers), communication can break down.
- *Principal:* To alleviate this TD, the information exchange process should be made explicit in procedural models and to relevant stakeholders.

5 EVALUATION

In this section we discuss our evaluation results, on the one hand we validated the findings of our human-based Technical Debt (TD) analysis in semi-structured interviews with with domain experts from the respective companies. Furthermore, we conducted an online survey that was completed from ten participants from a relevant production system industry consortium.

5.1 Expert Interviews

The expert interviews with the company partners had two aims: First to validate our findings concerning identified TD and to find out whether this concept is a viable communication means for engineering domain experts. Secondly, we wanted to find out more about the reasons for TD (causes) and the visible manifestations of TD in the organisations (effects).

5.1.1 Expert Interviews Context. A interview guideline was prepared[2]. Both interviews lasted around ninety minutes and were recorded in a textual protocol, as well audio-visual. The first interview was with a senior simulation engineer in company A with 20 years work experience in Production Systems Engineering (PSE), eight years in simulation and four years in his current position. The second interview was with two senior project managers in company B , one with seven years work experience in PSE and the other one over 30 years. Two researchers from our research group were involved in analysing and coding relevant phrases from the interviews to reduce bias.

Technical Debt as a Communication Tool in Production Systems Engineering. In both interviews the domain experts agreed that TD could be an effective tool for starting to communicate and negative long term effects in the design of PSE to management and engineers. In overall, the domain experts also stated that there are anti-patterns occurring in projects, which are posing challenges and limiting possibilities for present and future projects. However, internal communication on these issues is not yet established, since software-based approaches were integrated into the processes over time without being aware about consequences and adapting these processes.

Technical Debt Causes in Production Systems Engineering. Major reasons for accumulating TD are time and market pressure, which were mentioned in both interviews. The parallel development and insufficient tool-support for multi-version data instances can also lead to challenges in the further design and development of PSE. One participant noted that *Everybody copies whatever he can find.* Another one stated: *In order to receive orders or contracts, companies have to take out TD in order to succeed against competitors.* One could argue, that this is *prudent TD* according to [13]. However, it became clear, that this debt is not entirely prudent TD: Engineers in the production domain do not always have the needed software (engineering) and data modeling background and knowledge to be able to predict future negative effects resulting from current technology and modeling decisions. Furthermore, engineering tools are specialised on providing the possibilities for editing discipline-specific properties, but are often not designed for cross-disciplinary exchange and thus, limiting engineers in their capabilities to exchange data. Another challenge is that there are multiple degrees of longevity of software code development: Scripts or solutions which are only designed for local purposes, are later used in more global contexts, but are difficult to maintain or extend. Such solutions can then result in glue code or pipelines jungles, described in [36].

Technical Debt Effects in Production Systems Engineering. TD in this domain is mostly experienced by data consumers, such as software engineers joining the round-trip engineering process later in the project, who suffer from higher data extraction and integration efforts. But also future projects, who would need information from old documentation and lessons learned suffer, if data models are not well documented or information cannot be found. Interviewees also mentioned that onsite staff can also be negatively influenced during commissioning and operation due to limited or missing technical documentation or higher effort for training.

Technical Debt Management in Production Systems Engineering. So far, only limited awareness exists for TD in the data exchange and Software Engineering (SE). Thus, there is also no strategic repayment and management approach and debt is not resolved in a systematic way. In the worst case only more debt is accumulated over time. Since stakeholders later in the process (data consumers, software engineers, onsite staff) suffer from the effects of TD, there are only limited resources to change the overall process and few improvement initiatives span over multiple workgroups.

5.2 Survey

The goal of the online survey was to find out whether the identified TD was also applicable to other companies according to domain experts. Topic of the survey was the discussed framework in Section 4 and the experienced results of existing TD.

Survey Context and Demographics. The survey was available online through Google Forms[3] and sent out via a newsletter of an industrial consortium. Ten domain experts completed the survey during a period of one month. Companies ranged from automation technology, to electrical engineering, manufacturing automotive and PSE. The majority of companies had more than 1,000 employees, the rest

[2] https://cdl-sqi.ifs.tuwien.ac.at/techdebt2020/interview-guideline

[3] https://cdl-sqi.ifs.tuwien.ac.at/techdebt2020/survey-guideline

were small to medium size companies. 80% of the participants had worked more than five years of experience in their work field and thus, had extensive experience in PSE. The most common role of participants are engineers, three are project leaders. The previous expert interviews for validating the TD framework were conducted with mostly project management, so the validation by engineers, who are suffering from TD was extremely valuable for the survey.

Survey Results. We discuss the results concerning applicability of the TD categories and TD effects.

Technical Debt Categories. In Figure 3 we see how many participants agreed that the respective TD item is applicable to their exemplary project. For more than the majority of respondents the following items were relevant in their last project: artefact-based data exchange, incompatible description languages and inadequate description language mapping. Especially, artefact- based data exchange was common to many participants, eight of the sample agreed that they exchanged data that way. But also the category, weak versioning of data instances and inadequate semantic descriptions were relevant to four participants. Data model documentation was the least relevant with agreement of three respondents. One hypothesis could be that standards are adhered or alleviating measures have already been developed in these companies to address the need for versioning and semantic descriptions. Due to the limited scope of a survey, it was not possible to ask for further information. Additional interviews with more interviews could get more insights into these topics.

Technical Debt Effects. Answer possibilities included uncertainties about exchanged data, unplanned rework efforts and mistakes or wrong assumptions about the data, other impact (optional to provide a textual reply) and not applicable. Reported effects for artefact-based data exchange, included uncertainties about exchanged data, unplanned rework efforts and mistakes or wrong assumptions about the data. One participant also stated that errors occurred due to wrong input I/O specification or naming. For all TD items, respondents stated uncertainties about the exchanged data during the engineering process.

This is unfortunate, assuming that the quality of system designs is high delivered by domain experts. However, due to the identified challenges in the data exchange process, errors and mistakes can easily occur due to misunderstandings.

Technical Debt Causes. Since the design of an online survey only provides limited space to ask more detailed questions, we did not include questions about TD causes. However, the investigation of such causes would be of high interest and require future work to analyse TD in the data exchange process in more detail.

6 DISCUSSION

Data exchange is an essential task for the design and engineering of production systems. Considering the developments towards cyber-physical systems and industrial internet of things the extraction and integration of valuable knowledge will only become more important. Therefore, analysis and improvement measures to improve the data exchange and integration could have major impact in the future success of industrial companies and could also provide valuable lessons for other domains and industries. Since various challenges were similar to issues in Software Engineering, we applied Technical Debt to the Production Systems Engineering domain to communicate and visualise these difficulties.

In the following we report our experiences from improvement initiatives based on our findings and the ongoing research project with industrial partners:

Cultural Changes. One main finding from our workshops was that communication between data consumer and data provider can be improved: Stakeholders from different workgroups usually exchange data, but tend do not communicate enough and if at all through unstructured communication channels. Data providers were not usually aware about consumer needs regarding engineering data, although in some cases the requirements could be met without major changes. In the moment, the engineering data exchange is mainly provider-driven. Similar to the findings in [25], we reached the conclusion that individual teams cannot be selected for TD management because of conflicting goals of workgroups such as saving time or higher output. Oecker et al. also discuss the importance of increasing the awareness for TD in [28]. Therefore, we propose company-wide or at least workgroup-spanning improvement initiatives.

Enhancing Visualisation Capabilities for Model Engineering. The structure of engineering data is complex: Components are depicted in hierarchical form, encoding a vast amount of essential knowledge, such as relationships, parameters and dependencies to various disciplines and other components of the system. Extensible Markup Language (XML)-based languages, such as AutomationML (AML) aim to simplify and support the data exchange in engineering contexts, however the readability and efficient processing of such data is still questionable in text-based editors. Humans can only process a limited amount of information, however applying information visualisation means can support more efficient processing of components [24].

Vogel-Heuser et al. suggest to implement tools for visualisation of cross-discipline changes and adherence to requirements [41]. Jäger et al. [17] aimed to visualise technical dependencies in PSE using cause-effect-diagrams. The authors conclude, that not all information is always needed for specific tasks, and therefore the facilitation to curate and select information is important. Based on industrial requirements we designed a proof-of-concept prototype of a graph-based model inspection tool [32]. Its goal is to facilitate easier detection of changes and dependencies by visualising the hierarchical engineering information in space-efficient nodes in comparison to text-based model editors.

Test-Driven Model Development. Plant and systems model are oftentimes the starting point for systems engineer to commence the design phase. These models are either based on previous designs or from other templates adapted to new specifications or requirements. Due to the variety and heterogeneity of tools, there are no formal verification and validation of designs, but rather tedious manual error search.

On the other hand, Model Engineering and Test-Driven Development (TDD) [1] have both been extensively researched in SE contexts. In [20] the authors propose a meta-modeling framework

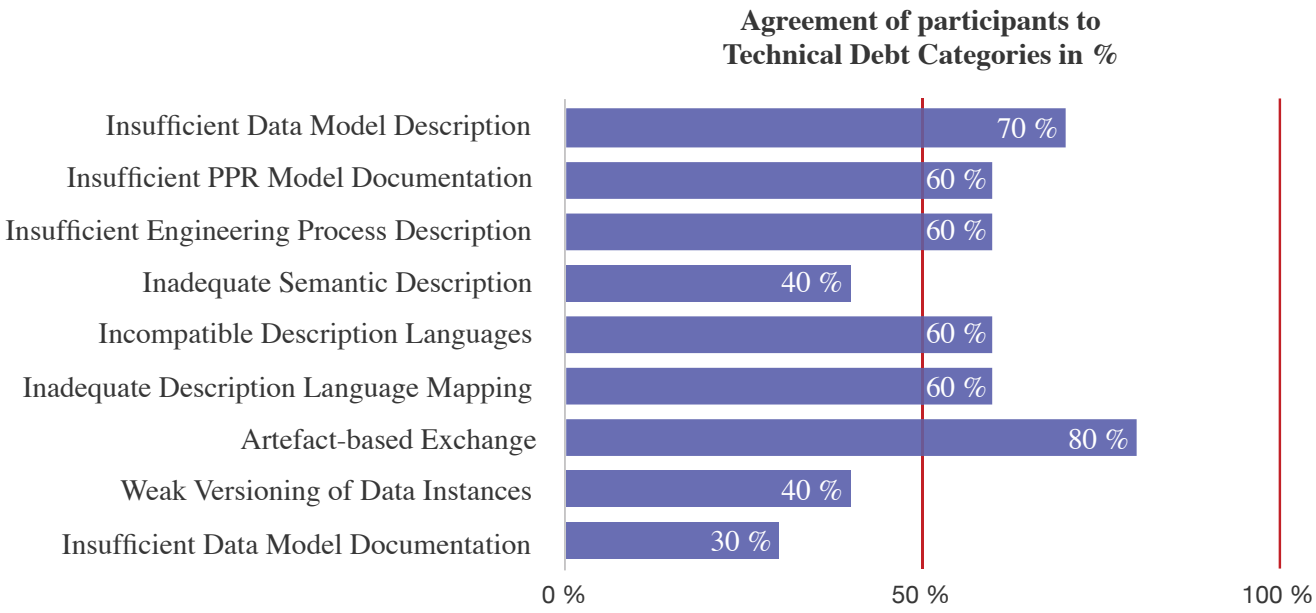

Figure 3: Results of survey regarding Technical Debt categories (n = 10)

in engineering by examples to reduce errors and ensure consistency for designed models. In cooperation with a modeling expert, we designed a testing pipeline for validating and verifying AML instance models in a systematic way [31]. The advantage is that, fewer manual error detection and resolution has to be done after specifying constraints and test cases for the respective project configuration. Domain experts also do not necessarily specific domain modeling expertise to specify test cases.

6.1 Limitations

The TD analysis was human-based due to the heterogeneity and non-automatic processing of engineering artefacts, this approach can introduce researcher bias. The small sample size for both evaluations expert interviews from two companies and response rate of the survey (n= 10) can be limiting for the significance of the results and the validation of the framework. However, the availability of industrial data in this particular domain is low and the initial survey evaluation indicate that the identified TD is relevant for other companies as well. Also the mix of roles project managers in interview setting versus majority of engineers in the survey context provides an improved description of the engineering context. The selection of companies was due to the context of a research project and thus, can introduce availability bias concerning the companies and experts at hand. However, results to other studies share similarities [10]. Another threat to validity could be that TD is yet not common in the domain and the complexity of topic questioned in a survey. Therefore misunderstandings to the questions asked in the survey could be misleading for participants. Further evaluation with interviews of a bigger sampler size could help to remove bias and validate our results regarding TD in PSE in the context of engineering data exchange.

7 CONCLUSION AND FUTURE WORK

We have presented the findings of a cross-case case study as a Technical Debt (TD) framework for engineering data exchange in Production Systems Engineering (PSE). Through qualitative analysis, we explored possible causes and effects and through an online survey we evaluated whether the identified TD items in data model, knowledge representation, exchange process are also applicable to other companies in the industrial domain. The framework focused on specific aspects such as knowledge representation and description languages, and might not be complete. However, we regard the state of the framework as a working hypothesis and a model to improve engineering data exchange and the application of Software Engineering (SE) methods in PSE. We presented different TD management strategies ranging from improving cross-discipline communication and repayment, to incorporating established approaches from SE such as model engineering and Test-Driven Development (TDD), which showed promising results for the discussed areas inflicting TD.

However, our improvement initiatives and the underlying TD framework are still at an early stage and should be researched in more detail to ensure increased validity and further learnings.

Tooling is an important aspect in managing the complexity of engineering data exchange, however it must be noted that appropriate tools have to be devised with much consideration to the needs of all stakeholders and their most important tasks. Complicating the tool chain without bringing much improved functionality could add even more TD to the overall process. Quantification of TD is a complex topic and should be also researched in the industrial context regarding the user perspective.

ACKNOWLEDGMENTS

The COMET center SBA Research (SBA-K1) is funded within the framework of COMET, Competence Centers for Excellent Technologies by BMVIT, BMDW, and the federal state of Vienna, managed by the FFG. Moreover, the financial support by the Christian Doppler Research Association, the Austrian Federal Ministry for Digital & Economic Affairs and the National Foundation for Research, Technology and Development is gratefully acknowledged.

REFERENCES

[1] Dave Astels. 2003. *Test driven development: A practical guide.* Prentice Hall Professional Technical Reference.

[2] Salman Azhar. 2011. Building information modeling (BIM): Trends, benefits, risks, and challenges for the AEC industry. *Leadership and management in engineering* 11, 3 (2011), 241–252.

[3] Terese Besker, Antonio Martini, Jan Bosch, and Matthias Tichy. 2017. An investigation of technical debt in automatic production systems. In *Proceedings of the XP2017 Scientific Workshops*. ACM, 6.

[4] Stefan Biffl, Fajar J. Ekaputra, Arndt Lüder, Johanna-Lisa Pauly, Felix Rinker, Laura Waltersdorfer, and Dietmar Winkler. 2019. Technical Debt Analysis in Parallel Multi-Disciplinary Systems Engineering. In *45th Euromicro Conference on Software Engineering and Advanced Applications, SEAA 2019, Kallithea-Chalkidiki, Greece, August 28-30, 2019*, Miroslaw Staron, Rafael Capilla, and Amund Skavhaug (Eds.). IEEE, 342–346. https://doi.org/10.1109/SEAA.2019.00059

[5] Stefan Biffl, Lukas Kathrein, Arndt Lüder, Kristof Meixner, Marta Sabou, Laura Waltersdorfer, and Dietmar Winkler. 2019. Software Engineering Risks from Technical Debt in the Representation of Product/ion Knowledge. In *The 31st International Conference on Software Engineering and Knowledge Engineering, SEKE 2019, Hotel Tivoli, Lisbon, Portugal, July 10-12, 2019*, Angelo Perkusich (Ed.). KSI Research Inc. and Knowledge Systems Institute Graduate School, 693–777. https://doi.org/10.18293/SEKE2019-037

[6] Stefan Biffl, Arndt Lüder, Felix Rinker, and Laura Waltersdorfer. 2019. Efficient Engineering Data Exchange in Multi-disciplinary Systems Engineering. In *Advanced Information Systems Engineering - 31st International Conference, CAiSE 2019, Rome, Italy, June 3-7, 2019, Proceedings (Lecture Notes in Computer Science)*, Paolo Giorgini and Barbara Weber (Eds.), Vol. 11483. Springer, 17–31. https://doi.org/10.1007/978-3-030-21290-2_2

[7] Stefan Biffl, Arndt Lüder, Felix Rinker, Laura Waltersdorfer, and Dietmar Winkler. 2019. *Engineering Data Logistics for Agile Automation Systems Engineering*. Springer, 187–225. https://doi.org/10.1007/978-3-030-25312-7_8

[8] Stefan Biffl, Arndt Lüder, Felix Rinker, Laura Waltersdorfer, and Dietmar Winkler. 2019. Quality Risks in the Data Exchange Process for Collaborative CPPS Engineering. In *17th IEEE International Conference on Industrial Informatics, INDIN 2019, Helsinki, Finland, July 22-25, 2019*. IEEE, 1217–1224. https://doi.org/10.1109/INDIN41052.2019.8972322

[9] Richard Brenner. 2019. Balancing resources and load: eleven nontechnical phenomena that contribute to formation or persistence of technical debt. In *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*. IEEE, 38–47.

[10] Quang Huan Dong, Felix Ocker, and Birgit Vogel-Heuser. 2019. Technical Debt as indicator for weaknesses in engineering of automated production systems. *Production Engineering* 13, 3-4 (2019), 273–282.

[11] Rainer Drath. 2009. *Datenaustausch in der Anlagenplanung mit AutomationML: Integration von CAEX, PLCopen XML und COLLADA*. Springer-Verlag.

[12] Gregory F. Dubos, David P. Coren, Alek Kerzhner, Seung H. Chung, and Jean-Francois Castet. 2016. Modeling of the flight system design in the early formulation of the Europa Project. In *2016 IEEE Aerospace Conference*. IEEE, 1–14.

[13] Martin Fowler. 2009. Technical Debt Quadrant.

[14] Yuepu Guo, Rodrigo Oliveira Spínola, and Carolyn Seaman. 2016. Exploring the costs of technical debt management–a case study. *Empirical Software Engineering* 21, 1 (2016), 159–182.

[15] Oliver Hummel, Holger Eichelberger, Andreas Giloj, Dominik Werle, and Klaus Schmid. 2018. A collection of software engineering challenges for big data system development. In *2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 362–369.

[16] Lorenz Hundt, Rainer Drath, Arndt Lüder, and Jörn Peschke. 2008. Seamless automation engineering with AutomationML®. In *2008 IEEE International Technology Management Conference (ICE)*. IEEE, 1–8.

[17] Tobias Jäger, Alexander Fay, Thomas Wagner, and Ulrich Löwen. 2011. Mining technical dependencies throughout engineering process knowledge. In *ETFA2011*. IEEE, 1–7.

[18] Lukas Kathrein, Arndt Lüder, Kristof Meixner, Dietmar Winkler, and Stefan Biffl. 2019. Production-aware analysis of multi-disciplinary systems engineering processes. In *21th International Conference on Enterprise Information Systems (ICEIS 2019)*.

[19] Zengyang Li, Paris Avgeriou, and Peng Liang. 2015. A systematic mapping study on technical debt and its management. *Journal of Systems and Software* 101 (2015), 193–220.

[20] Jesús J. López-Fernández, Jesús Sánchez Cuadrado, Esther Guerra, and Juan De Lara. 2015. Example-driven meta-model development. *Software & Systems Modeling* 14, 4 (2015), 1323–1347.

[21] Antonio Martini, Terese Besker, and Jan Bosch. 2018. Technical debt tracking: Current state of practice: A survey and multiple case study in 15 large organizations. *Science of Computer Programming* 163 (2018), 42–61.

[22] Antonio Martini and Jan Bosch. 2015. The danger of architectural technical debt: Contagious debt and vicious circles. In *2015 12th Working IEEE/IFIP Conference on Software Architecture*. IEEE, 1–10.

[23] Antonio Martini, Jan Bosch, and Michel Chaudron. 2014. Architecture technical debt: Understanding causes and a qualitative model. In *2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications*. IEEE, 85–92.

[24] Riccardo Mazza. 2009. *Introduction to information visualization*. Springer Science & Business Media.

[25] J. David Morgenthaler, Misha Gridnev, Raluca Sauciuc, and Sanjay Bhansali. 2012. Searching for build debt: Experiences managing technical debt at Google. In *2012 Third International Workshop on Managing Technical Debt (MTD)*. IEEE, 1–6.

[26] Andrew Nash, Daniel Huerlimann, Jörg Schütte, and Vasco Paul Krauss. 2004. RailML - a standard data interface for railroad applications. *WIT Transactions on The Built Environment* 74 (2004).

[27] Natalya F. Noy. 2004. Semantic Integration: A Sur vey Of Ontology-BasedApproaches. *ACM Sigmod Record* 33, 4 (2004), 65–70.

[28] Felix Ocker, Matthias Seitz, Marius Oligschläger, Minjie Zou, and Birgit Vogel-Heuser. 2019. Increasing Awareness for Potential Technical Debt in the Engineering of Production Systems. In *17th IEEE International Conference on Industrial Informatics (IEEE INDIN)*.

[29] Jan Oevermann. 2018. Semantic PDF Segmentation for Legacy Documents in Technical Documentation. *Procedia Computer Science* 137 (2018), 55–65.

[30] Julius Pfrommer, Miriam Schleipen, and Jürgen Beyerer. 2013. PPRS: Production skills and their relation to product, process, and resource. In *2013 IEEE 18th Conference on Emerging Technologies & Factory Automation (ETFA)*. IEEE, 1–4.

[31] Felix Rinker, Laura Waltersdorfer, and Stefan Biffl. 2020. Towards Test-Driven Model Development in Production Systems Engineering. In *22st International Conference on Enterprise Information Systems, ICEIS 2020, In Press*. In Press.

[32] Felix Rinker., Laura Waltersdorfer., Manuel Schüller., and Dietmar Winkler. 2020. Graph-based Model Inspection Tool for Multi-disciplinary Production Systems Engineering. In *Proceedings of the 8th International Conference on Model-Driven Engineering and Software Development - Volume 1: MODELSWARD,*. INSTICC, SciTePress, 116–125. https://doi.org/10.5220/0008990001160125

[33] Per Runeson and Martin Höst. 2009. Guidelines for conducting and reporting case study research in software engineering. *Empirical software engineering* 14, 2 (2009), 131.

[34] Günther Schuh, Jörg Hoffmann, Marie Gruber, and Violett Zeller. 2017. Managing IT complexity in the manufacturing industry. An agenda for action. *Journal on Systemics, Cybernetics and Informatics* 15, 2 (2017), 61–65.

[35] Ken Schwaber and Mike Beedle. 2002. *Agile software development with Scrum*. Vol. 1. Prentice Hall Upper Saddle River.

[36] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. In *Advances in neural information processing systems*. 2503–2511.

[37] Anton Strahilov and Holger Hämmerle. 2017. Engineering workflow and software tool chains of automated production systems. In *Multi-Disciplinary Engineering for Cyber-Physical Production Systems*. Springer, 207–234.

[38] Edith Tom, AybüKe Aurum, and Richard Vidgen. 2013. An exploration of technical debt. *Journal of Systems and Software* 86, 6 (2013), 1498–1516.

[39] Verein Deutscher Ingenieure. 2009. Richtlinie 3695: Engineering von Anlagen - Evaluieren und Optimieren des Engineerings. *VDI-Verlag, Düsseldorf* (2009).

[40] Verein Deutscher Ingenieure. 2009. Richtlinie 3695: Engineering von Anlagen - Evaluieren und Optimieren des Engineerings. *VDI-Verlag, Düsseldorf* (2009).

[41] Birgit Vogel-Heuser, Alexander Fay, Ina Schaefer, and Matthias Tichy. 2015. Evolution of software in automated production systems: Challenges and research directions. *Journal of Systems and Software* 110 (2015), 54–84. https://doi.org/10.1016/j.jss.2015.08.026

[42] Birgit Vogel-Heuser and Susanne Rösch. 2015. Applicability of technical debt as a concept to understand obstacles for evolution of automated production systems. In *2015 IEEE International Conference on Systems, Man, and Cybernetics*. IEEE, 127–132.

[43] Roel J. Wieringa. 2014. *Design science methodology for information systems and software engineering*. Springer.

[44] Jesse Yli-Huumo, Andrey Maglyas, and Kari Smolander. 2016. How do software development teams manage technical debt?–An empirical study. *Journal of Systems and Software* 120 (2016), 195–218.